

CNN Architectures, Transfer Learning & Data Augmentation

একটি সম্পূর্ণ, বিস্তারিত বাংলা নোট — Intuition থেকে Code পর্যন্ত

📄 LeNet · AlexNet · VGG · ResNet

🔗 Transfer Learning & Fine-Tuning

🔧 Data Augmentation কৌশল

📖 বিষয়সূচি

পর্ব ১ — CNN ARCHITECTURES

১.১ সহজ ভাষায় পরিচিতি — CNN Architecture কী?

১.২ বিস্তারিত ব্যাখ্যা — LeNet, AlexNet, VGG, ResNet

১.৩ Architecture তুলনামূলক চার্ট

১.৪ Python Code — সব Architecture Implement করা

১.৫ Visual Diagram — Layer Structure

১.৬ Pros, Cons & Memory Tricks

পর্ব ২ — TRANSFER LEARNING

২.১ Intuition — কেন Transfer Learning দরকার?

২.২ Feature Extraction vs Fine-Tuning পার্থক্য

২.৩ ধাপে ধাপে Fine-Tuning Process

২.৪ Python Code — VGG16 দিয়ে Transfer Learning

২.৫ Real-world Use Cases & Common Mistakes

পর্ব ৩ — DATA AUGMENTATION

৩.১ Intuition — কেন Data বাড়াতে হবে?

৩.২ সব Augmentation Techniques বিস্তারিত

৩.৩ Python Code — Albumentations & Keras

৩.৪ কোন Augmentation কখন ব্যবহার করবে?

৩.৫ Pros, Cons & Memory Tricks

Famous CNN Architectures

LeNet থেকে ResNet পর্যন্ত — Deep Learning-এর বিবর্তনের গল্প

১. সহজ ভাষায় পরিচিতি (Intuition)

কল্পনা করো তুমি একটি মাছ চিনতে শিখছ। প্রথমে তুমি শুধু "মাছের মতো আকৃতি" দেখো, তারপর আঁশ দেখো, তারপর চোখ ও মুখের ধরন দেখো। ধীরে ধীরে তুমি শুধু ছবি দেখেই বলতে পারো — "এটা রুই, এটা কাতলা।"

CNN (Convolutional Neural Network) ঠিক এইভাবেই কাজ করে। প্রতিটি layer একটু একটু করে ছবির **features** শেখে — প্রথমে edges (রেখা), তারপর shapes (আকৃতি), তারপর complex patterns (জটিল নকশা)।

* মূল প্রশ্ন

কিন্তু কোন CNN architecture সবচেয়ে ভালো? কেন নতুন নতুন architecture তৈরি হলো? প্রতিটি architecture কোন সমস্যা সমাধান করলো? — এই নোটে সব উত্তর আছে।

১৯৯৮ সাল থেকে ২০১৫ সালের মধ্যে চারটি landmark architecture তৈরি হয়েছে যা Deep Learning-এর ইতিহাস পরিবর্তন করে দিয়েছে:

১৯৯৮ — LENET

জন্ম হলো CNN-এর

Yann LeCun হাতের লেখা সংখ্যা চেনার জন্য তৈরি করলেন। মাত্র ৬০,০০০ parameter।

২০১২ — ALEXNET

Deep Learning-এর বিপ্লব

ImageNet Competition জিতলো। GPU ব্যবহার শুরু। Error ২৬% থেকে ১৫.৩%-এ নামলো।

২০১৪ — VGG

Simplicity is Power

শুধু ছোট ৩x৩ filter দিয়ে গভীরে গেলো। ১৩৮M parameter কিন্তু সহজ structure।

২০১৬ — RESNET

১৫২ Layer — Vanishing Gradient সমাধান

Skip Connection দিয়ে যেকোনো গভীর network train করা সম্ভব হলো। Error ৩.৫৭%।

২. বিস্তারিত ব্যাখ্যা (Deep Explanation)

LeNet-5

১৯৯৮

~৬১K Params

MNIST Pioneer

Yann LeCun তৈরি করেন। Bank চেক পড়ার জন্য ব্যবহার হতো। CNN-এর দাদাব্বর।

Input: 32×32 (Grayscale)

Conv layers: ২টি (5×5 filter)

Pooling: Average Pooling

Activation: Tanh / Sigmoid

FC layers: ৩টি

Output: 10 class (0-9 digit)

AlexNet

২০১২

~60M Params

ILSVRC 2012 Winner

Krizhevsky, Sutskever, Hinton তৈরি করেন। GPU ব্যবহার করে ৬ দিনে train।

Input: 224×224 (RGB Color)

Conv layers: ৫টি (11×11, 5×5, 3×3)

Pooling: Max Pooling

Activation: ReLU (নতুন!)

Regularization: Dropout + LRN

FC layers: ৩টি → 1000 class

VGG-16

২০১৪

~138M Params

ILSVRC 2014 Runner-up

Oxford University-র Simonyan ও Zisserman তৈরি করেন। Simple but deep।

Input: 224×224 (RGB)

Conv layers: ১০টি (সব 3×3!)

Pooling: Max Pooling (5টি)

Activation: ReLU সব জায়গায়

Key insight: 2×(3×3) = 5×5 receptive field

FC layers: ৩টি → 1000 class

ResNet-50

২০১৫

~25M Params

ILSVRC 2015 Winner

Microsoft-এর Kaiming He তৈরি করেন। Skip Connection দিয়ে বিপ্লব।

Input: 224×224 (RGB)

Layers: 50 (বা 101, 152)

Key innovation: Skip/Residual Connection

Batch Norm: প্রতিটি layer-এ

No dropout (BatchNorm করে কাজ)

Output: 1000 class, Top-5 error 3.57%

🔑 AlexNet-এর Key Innovations বিস্তারিত

LeNet থেকে AlexNet-এর সবচেয়ে বড় পার্থক্য হলো:

- **ReLU Activation:** আগে Tanh ব্যবহার হতো। ReLU training ৬ গুণ দ্রুত করে। কারণ ReLU-তে gradient সহজে flow করে।
- **GPU Training:** প্রথমবার ২টি NVIDIA GTX 580 GPU ব্যবহার — network-কে দুটি ভাগে ভাগ করে parallel training।
- **Dropout:** FC layer-এ 50% neurons randomly বন্ধ করে রাখে। Overfitting কমায়ে।

- Local Response Normalization (LRN):** একটি নিউরনের activation পাশের নিউরনগুলোকে suppress করে।
- Data Augmentation:** Image flip, crop করে dataset বড় করে।

🔑 VGG-এর Key Insight — ছোট Filter-এর জাদু

🔑 গুরুত্বপূর্ণ ধারণা

একটি **5x5 filter** এর সমান receptive field পাওয়া যায় দুটি **3x3 filter** stack করলে।
কিন্তু 5x5 তে parameters = 25, আর দুটি 3x3 তে parameters = 18 (28% কম!)।
তাহাড়া দুটি 3x3 মানে দুটি ReLU — বেশি non-linearity — model আরো powerful।

🔑 ResNet-এর Skip Connection — সবচেয়ে গুরুত্বপূর্ণ উদ্ভাবন

সমস্যা: Network যত গভীর হয়, gradient backward propagate করতে করতে ছোট হয়ে যায় — এটাই **Vanishing Gradient Problem**। ৫০+ layer হলে training প্রায় থেমে যায়।

সমাধান: ResNet বললো — layer-টি যা শিখুক না কেন, আমি input (x) সরাসরি output-এ যোগ করে দেবো।

$$H(x) = F(x) + x$$

যেখানে $F(x)$ = layer-গুলো কী শিখলো (residual), x = original input (skip connection)

এর ফলে worst case-এ layer কিছু না শিখলেও ($F(x)=0$), output = x, অর্থাৎ network খারাপ হয় না। Gradient এখন shortcut দিয়ে সরাসরি আসতে পারে।

📐 ৩. Math / Theory ও তুলনামূলক চার্ট

Architecture তুলনামূলক চার্ট

ARCHITECTURE	বছর	LAYERS	PARAMETERS	INPUT SIZE	TOP-5 ERROR	KEY INNOVATION
LeNet-5	১৯৯৮	৭	৬১K	32x32	—	প্রথম CNN
AlexNet	২০১২	৮	60M	224x224	15.3%	ReLU, GPU, Dropout
VGG-16	২০১৪	১৬	138M	224x224	7.3%	3x3 filter stack
ResNet-50	২০১৫	৫০	25M	224x224	3.57%	Skip Connection

🔑 মজার বিষয়

ResNet-50 মাত্র ২৫M parameters দিয়ে VGG-16 (138M) কে হারিয়ে দিয়েছে! Skip connection এত powerful।

Convolution Calculation

Output size বের করার formula:

$$\text{Output Size} = \lfloor (\text{Input} - \text{Filter} + 2 \times \text{Padding}) / \text{Stride} \rfloor + 1$$

উদাহরণ (LeNet, Layer 1):

- Input = 32×32 , Filter = 5×5 , Padding = 0, Stride = 1
- Output = $\lfloor (32 - 5 + 0) / 1 \rfloor + 1 = 28 \times 28$ ✓

8. Python Code — PyTorch Implementation

LeNet-5 Implementation

```
import torch
import torch.nn as nn

class LeNet5(nn.Module):
    def __init__(self, num_classes=10):
        super(LeNet5, self).__init__()

        # Feature Extraction অংশ
        self.features = nn.Sequential(
            # Layer 1:  $32 \times 32 \times 1 \rightarrow 28 \times 28 \times 6$  ( $5 \times 5$  filter, no padding)
            nn.Conv2d(1, 6, kernel_size=5),
            nn.Tanh(),
            # Pooling:  $28 \times 28 \rightarrow 14 \times 14$ 
            nn.AvgPool2d(kernel_size=2, stride=2),

            # Layer 2:  $14 \times 14 \times 6 \rightarrow 10 \times 10 \times 16$ 
            nn.Conv2d(6, 16, kernel_size=5),
            nn.Tanh(),
            # Pooling:  $10 \times 10 \rightarrow 5 \times 5$ 
            nn.AvgPool2d(kernel_size=2, stride=2),
        )

        # Fully Connected অংশ (Classifier)
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 5 * 5, 120), #  $400 \rightarrow 120$ 
            nn.Tanh(),
            nn.Linear(120, 84), #  $120 \rightarrow 84$ 
            nn.Tanh(),
            nn.Linear(84, num_classes), #  $84 \rightarrow 10$ 
        )

    def forward(self, x):
        x = self.features(x) # Feature extract করো
        x = self.classifier(x) # Classify করো
        return x

# Model তৈরি করো এবং দেখো
model = LeNet5(num_classes=10)
print(f"LeNet-5 Total Parameters: {sum(p.numel() for p in model.parameters()):,}")
# Output: LeNet-5 Total Parameters: 61,706
```

ResNet — Skip Connection Manually

```
class ResidualBlock(nn.Module):
    # ResNet-এর মূল building block
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()

        # Main path: দুটি Convolution
        self.conv1 = nn.Conv2d(in_channels, out_channels,
                                kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)
        self.conv2 = nn.Conv2d(out_channels, out_channels,
                                kernel_size=3, stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        # Skip Connection: যদি size না মেলে, project করো
```

```
self.shortcut = nn.Sequential()
if stride != 1 or in_channels != out_channels:
    self.shortcut = nn.Sequential(
        nn.Conv2d(in_channels, out_channels, kernel_size=1,
                    stride=stride, bias=False),
        nn.BatchNorm2d(out_channels)
    )

def forward(self, x):
    out = self.relu(self.bn1(self.conv1(x))) # প্রথম Conv
    out = self.bn2(self.conv2(out))         # দ্বিতীয় Conv
    out += self.shortcut(x)                 # ↗ Skip Connection: F(x) + x
    out = self.relu(out)
    return out

# Test করা
block = ResidualBlock(64, 64)
x = torch.randn(1, 64, 56, 56) # Batch=1, Channel=64, 56x56
out = block(x)
print(f"Input shape: {x.shape}")
print(f"Output shape: {out.shape}")
# Output: torch.Size([1, 64, 56, 56]) – shape একই থাকে!
```

Pretrained Models ব্যবহার করা (সবচেয়ে সহজ পদ্ধতি)

```
import torchvision.models as models
import torch.nn as nn

# ১ লাইনে Pretrained ResNet50 লোড করা
resnet50 = models.resnet50(weights='IMAGENET1K_V2')

# ১ লাইনে Pretrained VGG16 লোড করা
vgg16 = models.vgg16(weights='IMAGENET1K_V1')

# নিজের dataset-এর জন্য output layer পরিবর্তন করা
num_classes = 5 # ধরাও ৫ ধরনের ফুল চেনাতে চাই

# ResNet-50 এর শেষ layer বদলানো
resnet50.fc = nn.Linear(resnet50.fc.in_features, num_classes)
print(f"Modified ResNet-50 output layer: {resnet50.fc}")

# VGG-16 এর শেষ layer বদলানো
vgg16.classifier[6] = nn.Linear(4096, num_classes)
print(f"Modified VGG-16 output layer: {vgg16.classifier[6]}")
```

📊 ৩. Visual Diagram — Architecture Flow

LeNet-5 Architecture Flow (MNIST: 32x32 → 10 classes):

===== INPUT CONV1 POOL1 CONV2

POOL2 [32x32x1] → [28x28x6] → [14x14x6] → [10x10x16] → [5x5x16] (5x5 filter) (2x2 avg) (5x5 filter) (2x2 avg) 6 feature tanh 16 feature tanh maps maps FLATTEN FC1 FC2 OUTPUT [400] → [120] → [84] → [10] (tanh) (tanh) (softmax) 0,1,2...9

ResNet Residual Block – Skip Connection: =====

Input x | |-----| | (Shortcut) ▼ | Conv 3x3 | BatchNorm |

ReLU | | ▼ | Conv 3x3 | BatchNorm | | ▼ | (+) ←-----| | ReLU |

Output: H(x) = F(x) + x

Evolution of CNN Error Rate on ImageNet:

===== Year | Model | Top-5 Error |

Depth | Params ----- 1998 | LeNet-5 |

N/A (MNIST) | 7 | 61K 2012 | AlexNet | 15.3% | 2014 | VGGNet | 7.3%
2015 | ResNet-152 | 3.57% | (Human) | 5.1% ← 인간보다 정확!
ResNet exceeded human! Human Performance $\approx$ 5.1%



৬. Pros & Cons তুলনা

LeNet-5

✓ সুবিধা

- ✓ অত্যন্ত lightweight
- ✓ Simple, বোঝা সহজ
- ✓ CPU-তেও চলে
- ✓ শেখার জন্য perfect

✗ অসুবিধা

- ✗ Complex image handle করতে পারে না
- ✗ Tanh = slow training
- ✗ শুধু grayscale ছোট image
- ✗ Modern task-এ কাজ করে না

ResNet

✓ সুবিধা

- ✓ Vanishing gradient নেই
- ✓ যেকোনো depth সম্ভব
- ✓ State-of-the-art accuracy
- ✓ Transfer learning-এ best
- ✓ VGG থেকে কম parameter

✗ অসুবিধা

- ✗ Implementation জটিল
- ✗ Memory বেশি লাগে
- ✗ Training সময় বেশি
- ✗ Edge device-এ কঠিন



৭. Common Mistakes & Gotchas

- **Input size** ভুল করা: LeNet চায় 32×32 , AlexNet/VGG/ResNet চায় 224×224 । ভুল size দিলে error।
- **Channel** ভুল: LeNet grayscale (1 channel), বাকিগুলো RGB (3 channel) — মনে রাখো।
- **Parameter freeze** না করা: Pretrained model ব্যবহারে প্রথমে সব layer freeze করতে হয়।
- **Batch Normalization**-এর **mode**: Inference-এ `model.eval()` না দিলে BN ভুল কাজ করে।
- **VGG**-এর **FC layer** বড়: VGG-এর ৩টি FC layer মিলে ~ 1000 M parameter — memory অনেক লাগে।

🔑 মনে রাখার কৌশল: "লালু আলেক জইয়া রেজাল্ট"

LeNet → *AlexNet* → *VGG* → *ResNet* — প্রতিটি নতুন *Architecture* একটি নতুন সমস্যা সমাধান করেছে

LeNet = জন্ম | AlexNet = GPU+ReLU | VGG = Depth | ResNet = Skip Connection

Transfer Learning

Pre-trained model ব্যবহার করে নিজের কাজে লাগানো

১. সহজ ভাষায় পরিচিতি (Intuition)

ধরো তুমি বাংলাদেশে BUET থেকে Civil Engineering পাশ করেছ। এখন কানাডায় কোনো construction company তোমাকে hire করতে চায়। তোমাকে কি শূন্য থেকে engineering শিখতে হবে? না! তোমার পড়াশোনার **knowledge transfer** হবে।

Transfer Learning ঠিক এইভাবে কাজ করে। ImageNet-এ train করা একটি model ইতিমধ্যে শিখেছে — edge, texture, shape, color patterns। এখন তুমি এই knowledge নিয়ে নিজের specific task (যেমন: বাংলাদেশি ফুলের ছবি classify করা) এ ব্যবহার করতে পারো।

★ কেন TRANSFER LEARNING দরকার?

- ImageNet-এ train করতে লাগে হাজার হাজার GPU-hours এবং লক্ষ লক্ষ টাকা।
- তোমার কাছে মাত্র কয়েকশো ছবি আছে — এত কম data-তে scratch থেকে train সম্ভব নয়।
- Transfer Learning ব্যবহার করলে ঐ same result পেতে ৯০%+ কম data ও সময় লাগে।

২. দুটি পদ্ধতি — Feature Extraction vs Fine-Tuning

পদ্ধতি ১: Feature Extraction (Frozen Backbone)

Pretrained model-এর সব layer **freeze** করো (weights পরিবর্তন হবে না)। শুধু শেষের নতুন Classifier layer train করো।

Feature Extraction: 

পদ্ধতি ২: Fine-Tuning (Partial Unfreeze)

প্রথমে Feature Extraction করো। তারপর উপরের কিছু layer unfreeze করে খুব ছোট **learning rate** দিয়ে আবার train করো।

Fine-Tuning: 

কোনটা কখন ব্যবহার করবে?

পরিস্থিতি	পরামর্শ	কারণ
Data কম (< ১০০০) + Source task similar	Feature Extraction	Overfitting এড়াতে freeze করো
Data মাঝারি (১০০০-১০০০০) + Similar task	Feature Extraction + Fine-Tune শেষ কিছু layer	ধীরে ধীরে adapt করাও
Data বেশি (১০০০০+) + Different domain	Full Fine-Tuning	সব layer নতুন domain-এ adapt করাও
Source ও target task খুব আলাদা	Scratch থেকে Train	Pretrained knowledge কাজে নাও লাগতে পারে

৩. Transfer Learning-এর ধাপ (Step-by-Step)

১. Pretrained Model নির্বাচন করে

তোমার task-এর সাথে source task কতটা মিলে তা দেখো। Computer vision → ImageNet models (ResNet, VGG, EfficientNet)।

২. Output Layer পরিবর্তন করে

Original 1000-class output layer সরিয়ে তোমার task অনুযায়ী নতুন layer লাগাও (যেমন: ৫ class)।

৩. Layers Freeze করে

Pretrained layers-এ `requires_grad = False` সেট করে। শুধু নতুন layer train হবে।

৪. নতুন Classifier Train করে

তোমার dataset-এ বড় learning rate (1e-3) দিয়ে classifier train করো (৫-১০ epoch)।

৫. Fine-Tuning (Optional)

পেছের কিছু pretrained layer unfreeze করে খুব ছোট learning rate (1e-5) দিয়ে আবার train করো।

৪. Python Code — Complete Transfer Learning

```
import torch
import torch.nn as nn
import torchvision.models as models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from torchvision.datasets import ImageFolder

# — STEP 1: Data Preparation —
transform = transforms.Compose([
    transforms.Resize((224, 224)), # ResNet চায় 224x224
    transforms.RandomHorizontalFlip(), # Augmentation (পরে বিস্তারিত)
    transforms.ToTensor(),
```

```

transforms.Normalize(                                # ImageNet এর mean/std দিয়ে normalize
    mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225]
)
])

# তোমার dataset লোড করো (folder structure: data/train/class1/, data/train/class2/)
train_data = ImageFolder('data/train', transform=transform)
val_data   = ImageFolder('data/val', transform=transform)

train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
val_loader   = DataLoader(val_data, batch_size=32, shuffle=False)

# ——— STEP 2: Pretrained Model লোড ———
model = models.resnet50(weights='IMAGENET1K_V2')

# ——— STEP 3: সব Pretrained Layer Freeze করো ———
for param in model.parameters():
    param.requires_grad = False # এই layer train হবে না

# ——— STEP 4: নতুন Classifier যোগাও ———
num_classes = 5 # তোমার class সংখ্যা
model.fc = nn.Sequential(
    nn.Linear(model.fc.in_features, 256), # 2048 → 256
    nn.ReLU(),
    nn.Dropout(0.3), # Overfitting কমাতে
    nn.Linear(256, num_classes) # 256 → 5
)
# এখন শুধু model.fc-র parameters train হবে

# ——— STEP 5: Optimizer ও Loss ———
optimizer = torch.optim.Adam(
    model.fc.parameters(), # শুধু নতুন layer-এর parameters
    lr=1e-3
)
criterion = nn.CrossEntropyLoss()

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)

# ——— STEP 6: Training Loop ———
def train_epoch(model, loader, optimizer, criterion, device):
    model.train()
    total_loss, correct = 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad() # Gradient zero করো
        outputs = model(images) # Forward pass
        loss = criterion(outputs, labels) # Loss calculate
        loss.backward() # Backward pass
        optimizer.step() # Weights update

        total_loss += loss.item()
        _, predicted = outputs.max(1)
        correct += predicted.eq(labels).sum().item()

    acc = 100 * correct / len(loader.dataset)
    return total_loss / len(loader), acc

# Phase 1: Classifier train করো (5 epoch)
for epoch in range(5):
    loss, acc = train_epoch(model, train_loader, optimizer, criterion, device)
    print(f"Epoch {epoch+1}/5 | Loss: {loss:.4f} | Acc: {acc:.2f}%")

# ——— STEP 7: Fine-Tuning ———
# শেষের ২টি ResNet block unfreeze করো
for name, param in model.named_parameters():
    if 'layer4' in name or 'layer3' in name:
        param.requires_grad = True # এখন এই layers-ও train হবে

# Fine-tuning: ছোট learning rate
optimizer_ft = torch.optim.Adam([
    {'params': model.layer3.parameters(), 'lr': 1e-5},
    {'params': model.layer4.parameters(), 'lr': 1e-5},
    {'params': model.fc.parameters(), 'lr': 1e-4},
])

for epoch in range(5):
    loss, acc = train_epoch(model, train_loader, optimizer_ft, criterion, device)

```

```
print(f"Fine-tune Epoch {epoch+1}/5 | Loss: {loss:.4f} | Acc: {acc:.2f}%")
```



৫. Real-World Use Cases

- **Medical Imaging:** X-ray বা MRI থেকে রোগ detect করতে ResNet/DenseNet ব্যবহার হয়। MIT Medical School-এ diabetic retinopathy detection।
- **Agriculture:** পাতার রোগ detect করতে Plant Village dataset-এ ResNet fine-tune করা হয়েছে।
- **Industrial Defect Detection:** Factory-তে product-এর defect detect করতে। কম labeled data-তে কাজ করে।
- **Face Recognition:** FaceNet (Google), DeepFace (Facebook) — ImageNet pre-trained ব্যবহার করে।
- **Satellite Image:** Remote sensing — land use classification, building detection।



৬. Common Mistakes

- **Normalization** ভুলে যাওয়া: ImageNet models expect করে `mean=[0.485, 0.456, 0.406]` normalize। না করলে accuracy অনেক কম হবে।
- এ ক সাথে সব **unfreeze** করা: প্রথমেই সব layer unfreeze করলে large gradient এসে pretrained weights নষ্ট করবে ("catastrophic forgetting")।
- **Fine-tuning** এ বড় **learning rate**: Fine-tuning এ সর্বোচ্চ $1e-5$ ব্যবহার করে। বড় rate দিলে সব শেখা ভুলে যাবে।
- **BatchNorm mode**: Fine-tuning এ `model.eval()` না দিলে BN statistics আপডেট হয়ে যাবে।
- ভুল **pretrained model**: Medical image-এ ImageNet pretrained ভালো নাও হতে পারে। Domain যত কাছাকাছি তত ভালো।



মনে রাখার কৌশল: "ফ্রিজ রাখো, তারপর গরম করো"

প্রথমে Freeze করো (Feature Extraction) → তারপর ধীরে ধীরে Unfreeze করো (Fine-Tuning)

১ লাইনে সারসংক্ষেপ: "অন্যের জ্ঞান ধার নাও, নিজের কাজে লাগাও — সময় ও resource বাঁচাও"

Data Augmentation

Training data কৃত্রিমভাবে বাড়ানোর কৌশল

১. সহজ ভাষায় পরিচিতি (Intuition)

ধরো তুমি একটি শিশুকে "বিড়াল" চেনাচ্ছে। তুমি শুধু একটা সাদা বিড়ালের ছবি দেখালে সে শিখবে "সাদা জিনিসই বিড়াল।" কিন্তু যদি তুমি তাকে বিভিন্ন রঙের, বিভিন্ন আলোতে, বিভিন্ন কোণ থেকে বিড়ালের ছবি দেখাও — সে সত্যিকারের "বিড়াল" concept শিখবে।

Data Augmentation ঠিক এটাই করে। একটি ছবিকে বিভিন্নভাবে transform করে নতুন নতুন training sample তৈরি করে। Model আরো generalized হয়।

🔥 কেন দরকার?

Deep Learning model train করতে লক্ষ লক্ষ ছবি দরকার। কিন্তু বাস্তবে এত data পাওয়া কঠিন। Data Augmentation ছাড়া model **Overfitting** করে — training-এ ৯৯% accuracy কিন্তু test-এ ৬০%!

২. সব Augmentation Techniques বিস্তারিত



Horizontal Flip

ছবি আয়নার মতো উল্টে দাও। বিড়াল ডান দিক বা বাম দিক — দুটোই বিড়াল।



Rotation

ছবি ঘুরিয়ে দাও ($\pm 15^\circ$, $\pm 30^\circ$)। কিছু ছবিতে সাবধান — 6 vs 9 গুলিয়ে যেতে পারে।



Random Crop

ছবির random অংশ কেটে নাও। Object সবসময় কেন্দ্রে থাকে না।



Brightness/Contrast

আলোর পরিমাণ ও contrast বাড়ানো-কমানো। দিনে/রাত্রে/মেঘলা দিনে একই object।



Gaussian Noise

ছবিতে random noise যোগ করে। Real camera-তে সবসময় কিছু noise থাকে।

Zoom / Scale

ছবি কাছে বা দূর থেকে দেখার মতো zoom in/out। Object বিভিন্ন দূরত্বে থাকে।

Color Jitter

Hue, saturation পরিবর্তন। লাল আপেল একটু হলুদাভ হলেও আপেলই।

Cutout / Erasing

ছবির এক অংশ কালো rectangle দিয়ে ঢেকে দাও। Occluded object চেনানো।

MixUp / CutMix

দুটি ছবি mix করে। Advanced technique — competition-এ popular।

কোন Augmentation কোন কাজে?

TASK	RECOMMENDED AUGMENTATION	এ ডানো উচিত
General Image Classification	Flip, Crop, Color Jitter, Rotation	—
Medical Imaging (X-ray)	Rotation, Flip, Zoom	Vertical Flip, Color change
Digit / Text Recognition	Slight rotation, Brightness	Rotation >15°, Flip (6↔9 গুলিয়ে যাবে)
Object Detection	Flip, Crop (bbox সহ), Color	—
Satellite / Aerial	360° Rotation, Flip উভয়দিকে	—

৩. Online vs Offline Augmentation

দুটি পদ্ধতি

Online (On-the-fly): Training চলার সময় প্রতিটি batch-এ real-time augmentation। প্রতিটি epoch-এ ভিন্ন augmented version দেখে। Memory কম লাগে। — সবচেয়ে জনপ্রিয়

Offline (Static): আগেই augmented ছবি disk-এ save করে। Training দ্রুত হয় কিন্তু বেশি disk space লাগে।

৪. Python Code — Albumentations & Keras

পদ্ধতি ১: Keras/TensorFlow দিয়ে (সহজ)

```
import tensorflow as tf
from tensorflow.keras import layers

# Data Augmentation layer তৈরি করে (Model-এর মধ্যে থাকবে)
data_augmentation = tf.keras.Sequential([
    # Random horizontal flip (বাম-ডান উল্টে দাও)
    layers.RandomFlip('horizontal'),

    # ±10% rotation (ঘুরিয়ে দাও)
    layers.RandomRotation(0.1),

    # ±20% zoom
    layers.RandomZoom(0.2),

    # Brightness পরিবর্তন
    layers.RandomBrightness(0.2),

    # Contrast পরিবর্তন
    layers.RandomContrast(0.2),
], name='augmentation')

# Model-এ augmentation যোগ করে
inputs = tf.keras.Input(shape=(224, 224, 3))
x = data_augmentation(inputs) # Training-এ augment হবে
x = tf.keras.applications.ResNet50(
    include_top=False, weights='imagenet'
)(x, training=False) # Pretrained backbone
x = layers.GlobalAveragePooling2D()(x)
outputs = layers.Dense(5, activation='softmax')(x)

model = tf.keras.Model(inputs, outputs)

# Note: data_augmentation layer training=False এ automatically বন্ধ থাকে
# তাই validation/test এ আলাদা কিছু করতে হবে না
```

পদ্ধতি ২: Albumentations দিয়ে (PyTorch — Professional)

```
import albumentations as A
from albumentations.pytorch import ToTensorV2
import cv2
import numpy as np
from torch.utils.data import Dataset

# Training augmentation (aggressive)
train_transform = A.Compose([
    # Geometric transforms
    A.Resize(224, 224),
    A.HorizontalFlip(p=0.5), # ৫০% সম্ভাবনায় flip
    A.RandomRotate90(p=0.3), # 90° ঘোরানো
    A.ShiftScaleRotate( # Shift, Scale, Rotate একসাথে
        shift_limit=0.1,
        scale_limit=0.15,
        rotate_limit=20,
        p=0.5
    ),

    # Color transforms
    A.ColorJitter( # Color পরিবর্তন
        brightness=0.2,
        contrast=0.2,
        saturation=0.2,
        hue=0.1,
        p=0.5
    ),

    # Noise & Blur
    A.OneOf([ # এর মধ্যে একটি random apply
        A.GaussNoise(var_limit=(10, 50)), # Gaussian noise
        A.GaussianBlur(blur_limit=3), # Blur
        A.MotionBlur(blur_limit=3), # Motion blur
    ], p=0.3),
```



```

# Cutout (Advanced)
A.CoarseDropout(                                # Random rectangle কালো করে
    max_holes=8, max_height=32, max_width=32,
    p=0.3
),

# Normalize এবং Tensor convert
A.Normalize(mean=[0.485, 0.456, 0.406],
             std=[0.229, 0.224, 0.225]),
ToTensorV2()
])

# Validation augmentation (minimal - শুধু resize ও normalize)
val_transform = A.Compose([
    A.Resize(224, 224),
    A.Normalize(mean=[0.485, 0.456, 0.406],
                 std=[0.229, 0.224, 0.225]),
    ToTensorV2()
])

# Custom Dataset
class FlowerDataset(Dataset):
    def __init__(self, image_paths, labels, transform=None):
        self.image_paths = image_paths
        self.labels       = labels
        self.transform     = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        # ছবি পাঠা (BGR → RGB convert করে)
        image = cv2.imread(self.image_paths[idx])
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        label = self.labels[idx]

        # Augmentation apply করে
        if self.transform:
            augmented = self.transform(image=image)
            image = augmented['image']

        return image, label

# Dataset তৈরি করে
# train_dataset = FlowerDataset(train_paths, train_labels, train_transform)
# val_dataset   = FlowerDataset(val_paths, val_labels, val_transform)
print("Albumentations pipeline ready!")
print(f"Train transforms: {len(train_transform.transforms)} steps")

```

MixUp Augmentation (Advanced)

```

import numpy as np
import torch

def mixup_data(x, y, alpha=0.2):
    """দুটি ছবি blend করে নতুন training sample তৈরি"""

    # Beta distribution থেকে mixing ratio নাও
    lam = np.random.beta(alpha, alpha) if alpha > 0 else 1

    batch_size = x.size(0)
    # Random shuffle করে
    index = torch.randperm(batch_size)

    # দুটি ছবি mix করে:  $\lambda \times \text{img1} + (1-\lambda) \times \text{img2}$ 
    mixed_x = lam * x + (1 - lam) * x[index, :]

    y_a, y_b = y, y[index]
    return mixed_x, y_a, y_b, lam

def mixup_criterion(criterion, pred, y_a, y_b, lam):
    """Mixed label-এর জন্য Loss calculate"""
    return lam * criterion(pred, y_a) + (1 - lam) * criterion(pred, y_b)

# Training loop-এ ব্যবহার:

```


Data Augmentation Pipeline:

Original Image [একটি বিড়ালের ছবি] | | ▼ Augment 1 Augment 2
▼ ... ↔ Flip ↻ Rotate 15° ✕ Crop ✱ Bright [বিড়াল ↔] [কিছুটা হেলানো] [বাম অংশ] [উজ্জ্বল] | | |
| | | | ▼ Training Dataset (৫x বড় হয়ে গেলো!) | ▼
Model শেখে সব variation-এ → Better Generalization! → Overfitting কমে!

Augmentation এর আগে ও পরে:

Without Augmentation: Training Acc: 95% Training Acc: 90% Validation Acc: 62% Validation Acc: 88% Gap (Overfit): 33% Gap: 2% ← ✓ Much Better!

- **AlexNet (2012):** প্রথমবারের মতো Data Augmentation systematic ভাবে ব্যবহার করেছিল — random crop, horizontal flip, color jitter । এটাই deep learning জয়ের অন্যতম কারণ ।
- **Self-Driving Car (Tesla, Waymo):** Rain, fog, night simulation augmentation করে rare driving conditions-এ robust করা হয় ।
- **Medical Imaging:** COVID-19 X-ray dataset-এ augmentation ছাড়া model ভালো করেনি । Rotation ও flip দিয়ে ৩x data বড় করা হয়েছিল ।
- **Kaggle Competitions:** প্রায় সব winning solution-এ Albumentations ব্যবহার করা হয় । TTA (Test Time Augmentation) দিয়ে inference accuracy ১-২% বাড়ানো হয় ।
- **Agriculture:** Plant disease detection-এ augmentation ছাড়া accuracy 41%, augmentation দিয়ে 89% ।

✓ ସୁବିଧା

- ✓ Overfitting উন্মেষযোগ্যভাবে কমায়ে
- ✓ কম data-তে ভালো accuracy
- ✓ Model robust হয় (real world ready)
- ✓ Regularization-এর কাজ করে
- ✓ প্রায় free — extra data collect করতে হয় না
- ✓ Albuementations দিয়ে implement সহজ

✕ ଅସୁବିଧା

- X** Training slow হয় (on-the-fly এ)

✗ ভুল augmentation label নষ্ট করে (digit 6→9)

✗ Bias propagate হয় original data থেকে

✗ Hyperparameter tuning দরকার

✗ নতুন information যোগ হয় না

৮ . Related Topics

আগে জ্ঞান দরকার	পরে শেখার বিষয়
CNN basics (Convolution, Pooling)	EfficientNet, Vision Transformer (ViT)
Overfitting / Underfitting concept	AutoAugment / RandAugment (automatic policy)
Backpropagation & Gradient	GAN দিয়ে Synthetic Data Generation
Python, NumPy, PyTorch/TF basics	Contrastive Learning (SimCLR, MoCo)

৯ . Common Mistakes

- **Validation data-তে augmentation:** Validation ও test data শুধু resize ও normalize করে। কোনো augmentation নয়।
- **Label-breaking augmentation:** Digit recognition-এ vertical flip করলে 6 হয়ে যায় 9। Task বুঝে augmentation বেছে নাও।
- অতিরিক্ত **augmentation:** সব transformation একসাথে apply করলে ছবি distorted হয়। 3-5টির বেশি না।
- **Augmentation order:** Geometric transform আগে, তারপর color। Normalize সবার শেষে।
- **Reproducibility:** Random seed set না করলে experiment reproduce করা যাবে না।

🧠 মনে রাখার কৌশল: "বাস্তব জীবনে যা হয়, **augment** এ তাই করে"

সূর্যের আলো পরিবর্তন হয় → **Brightness** | ক্যামেরা হেলায় → **Rotation** | দূরে থাকে → **Scale**

১ লাইনে সারসংক্ষেপ: "একটি ছবি → অনেক variation → Model দেখে সব → Real world-এ চমৎকার কাজ করে"

🧠 ১০ . ছড়ান্ত সারসংক্ষেপ — তিনটি **Topic** একনজরে

TOPIC	মূল সমস্যা সমাধান	মূল কৌশল	সেরা TOOL
CNN Architectures	Image-এ কী আছে চেনা	LeNet→AlexNet→VGG→ResNet evolution	PyTorch torchvision
Transfer Learning	কম data-তে ভালো model	Freeze → Classifier train → Fine-tune	Keras, Hugging Face

Data	Overfitting কমানো	Flip, Rotate, Crop, Color, Noise	Albumentations,
Augmentation			Keras

🔗 তিনটি একসাথে ব্যবহার করে!

সেরা result পেতে তিনটি technique একসাথে ব্যবহার করো:

১. **ResNet50/VGG16** (best architecture) + ২. **Transfer Learning** (কম data-তে) + ৩. **Data Augmentation** (overfitting কমাতে) = Production-ready Model!